

Name : Moshitha Kolanjiyappan

Register Number : 192421205

CSA0603 : Design and Analysis of Algorithms

CO2-AT3-Debugging and Optimization Task

Q. Debugging Linear Search Code (Incorrect Return Outside Logic) The following Linear Search implementation gives incorrect results because the return statement is misplaced and causes confusion in unsuccessful search cases. Analyze the control flow carefully and identify why the function may produce misleading output. Apply debugging to correct the search logic and ensure proper reporting when the target is found or absent. Optimize the implementation for readability and maintainability. Analyze the time complexity of the corrected algorithm. Rewrite the clean and corrected version with suitable comments.

```
def linear_search(arr, key):
```

```
    found = False
```

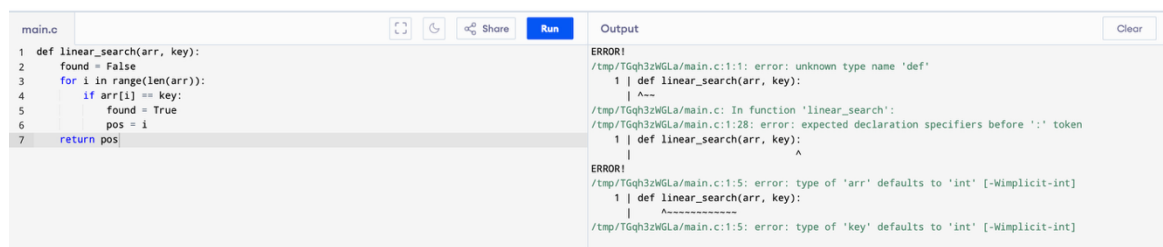
```
    for i in range(len(arr)):
```

```
        if arr[i] == key:
```

```
            found = True
```

```
            pos = i
```

```
    return pos
```



The screenshot shows a code editor with a file named 'main.c'. The code defines a function 'linear_search' that takes an array 'arr' and a 'key' as input. It initializes 'found' to False and iterates through the array. If a match is found, it sets 'found' to True and 'pos' to the current index 'i'. Finally, it returns 'pos'. The 'Run' button is highlighted. The output window shows several error messages: 'error: unknown type name 'def'', 'error: expected declaration specifiers before ':' token', and two 'error: type of 'arr' defaults to 'int' [-Wimplicit-int]' and 'error: type of 'key' defaults to 'int' [-Wimplicit-int]'.

```
main.c
1 def linear_search(arr, key):
2     found = False
3     for i in range(len(arr)):
4         if arr[i] == key:
5             found = True
6             pos = i
7     return pos

Output
ERROR!
/tmp/TGqh3zWGLa/main.c:1:1: error: unknown type name 'def'
1 | def linear_search(arr, key):
  | ^~~
/tmp/TGqh3zWGLa/main.c: In function 'linear_search':
/tmp/TGqh3zWGLa/main.c:1:28: error: expected declaration specifiers before ':' token
1 | def linear_search(arr, key):
  |                        ^
ERROR!
/tmp/TGqh3zWGLa/main.c:1:5: error: type of 'arr' defaults to 'int' [-Wimplicit-int]
1 | def linear_search(arr, key):
  |     ^~~~~~
/tmp/TGqh3zWGLa/main.c:1:5: error: type of 'key' defaults to 'int' [-Wimplicit-int]
```

pos is not initialized

```
main.c
1 def linear_search(arr, key):
2     found = False
3     pos = -1
4     for i in range(len(arr)):
5         if arr[i] == key:
6             found = True
7             pos = i
8     return pos

Output
ERROR!
/tmp/UDqLUEJ1bq/main.c:1:1: error: unknown type name 'def'
1 | def linear_search(arr, key):
  | ^~~
/tmp/UDqLUEJ1bq/main.c: In function 'linear_search':
/tmp/UDqLUEJ1bq/main.c:1:28: error: expected declaration specifiers before '::' token
1 | def linear_search(arr, key):
  | ^~~~~~
/tmp/UDqLUEJ1bq/main.c:1:5: error: type of 'arr' defaults to 'int' [-Wimplicit-int]
1 | def linear_search(arr, key):
  | ^~~~~~
/tmp/UDqLUEJ1bq/main.c:1:5: error: type of 'key' defaults to 'int' [-Wimplicit-int]

=== Code Exited With Errors ===
```

Explanation:

If the key is not found, pos is never created. So return pos gives an error. initialized pos = -1 before the loop. If the element is not found, the function returns -1 instead of giving an error.

Unnecessary found Variable

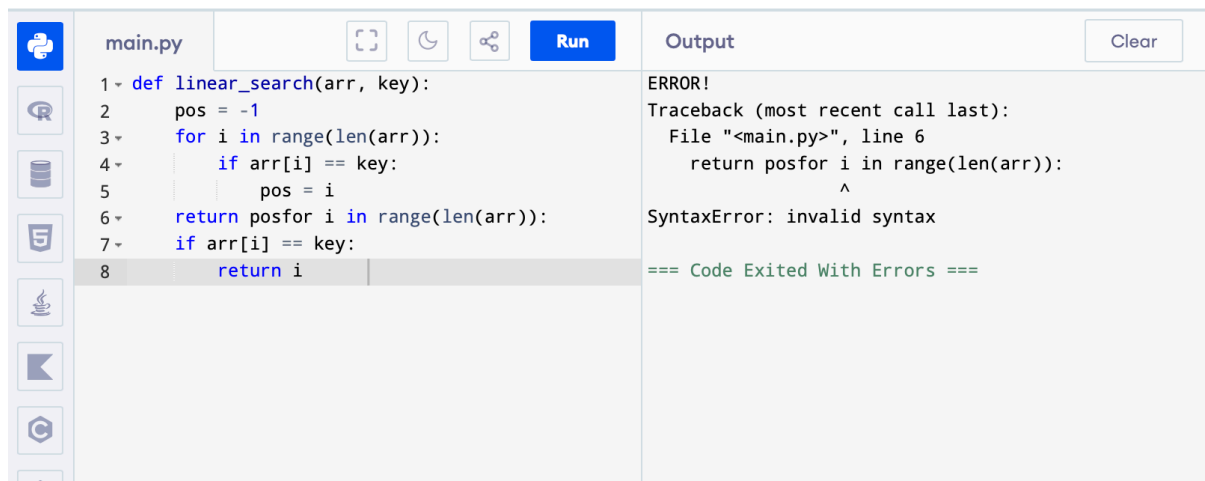
```
main.py
1 def linear_search(arr, key):
2     found = False
3     for i in range(len(arr)):
4         if arr[i] == key:
5             found = True
6             pos = i
7     return pos

Output
=== Code Execution Successful ===
```

Explanation:

The variable found is assigned but never used. It increases code complexity unnecessarily. The value of pos itself is enough to determine whether the element was found.

Loop Continues After Finding the Element



The screenshot shows a Python IDE with a file named `main.py`. The code defines a function `linear_search(arr, key)` with the following lines:

```
1 def linear_search(arr, key):
2     pos = -1
3     for i in range(len(arr)):
4         if arr[i] == key:
5             pos = i
6         return pos for i in range(len(arr)):
7         if arr[i] == key:
8             return i
```

The IDE highlights a syntax error on line 6. The output pane shows the following error message:

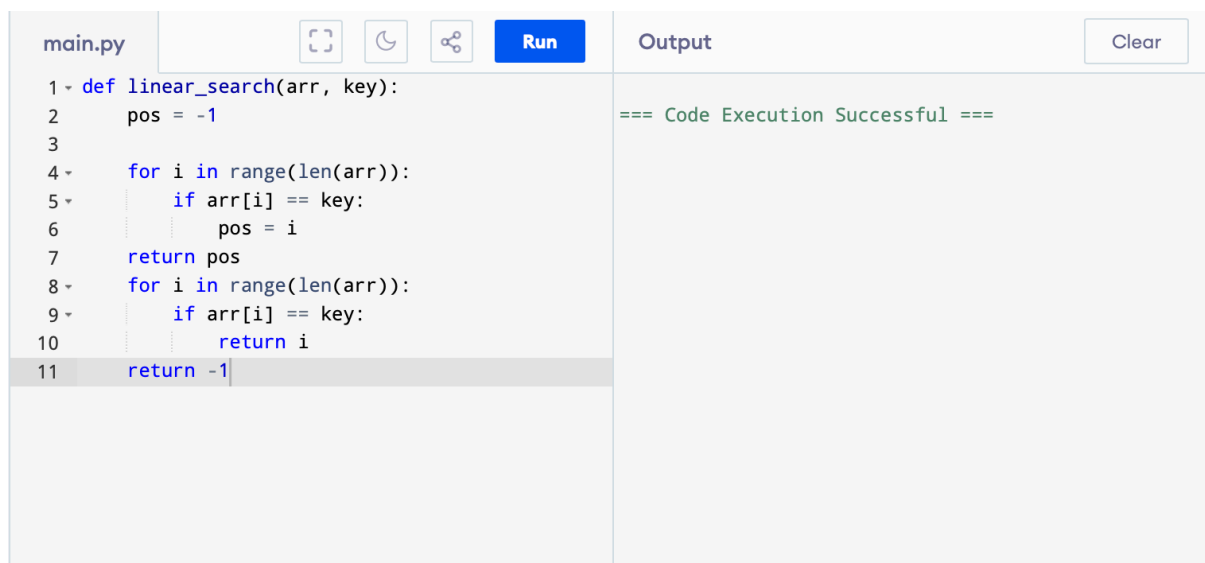
```
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 6
    return pos for i in range(len(arr)):
                ^
SyntaxError: invalid syntax

=== Code Exited With Errors ===
```

Explanation:

Even after finding the element, the loop continues checking the remaining elements. This causes unnecessary iterations. `return i` immediately after finding the element. This stops the search early and improves efficiency.

Incorrect Return Statement



The screenshot shows a Python IDE with a file named `main.py`. The code defines a function `linear_search(arr, key)` with the following lines:

```
1 def linear_search(arr, key):
2     pos = -1
3
4     for i in range(len(arr)):
5         if arr[i] == key:
6             pos = i
7     return pos
8     for i in range(len(arr)):
9         if arr[i] == key:
10            return i
11    return -1
```

The IDE shows the code execution was successful. The output pane displays the following message:

```
=== Code Execution Successful ===
```

Explanation:

Two statements are written on the same line. This causes a `SyntaxError`.

Optimized Code : Removed unnecessary variables, returned immediately when the key is found, and used `-1` when not found. This improves readability and efficiency.

main.py	Run	Output
<pre>1 def linear_search(arr, key): 2 pos = -1 3 for i in range(len(arr)): 4 if arr[i] == key: 5 pos = i 6 return pos 7 for i in range(len(arr)): 8 if arr[i] == key: 9 return i 10 return -1 11 arr = [12, 25, 8, 19, 30] 12 key = 50 13 14 result = linear_search(arr, key) 15 16 if result != -1: 17 print("Element found at index:", result) 18 else: 19 print("Element not found")</pre>		Element not found === Code Execution Successful ===

main.py	Run	Output
<pre>1 def linear_search(arr, key): 2 pos = -1 3 for i in range(len(arr)): 4 if arr[i] == key: 5 pos = i 6 return pos 7 for i in range(len(arr)): 8 if arr[i] == key: 9 return i 10 return -1 11 arr = [12, 25, 8, 19, 50] 12 key = 50 13 14 result = linear_search(arr, key) 15 16 if result != -1: 17 print("Element found at index:", result) 18 else: 19 print("Element not found")</pre>		Element found at index: 4 === Code Execution Successful ===

Algorithm

1. Initialize the array as {12, 25, 8, 19, 50}.
2. Set the key to 50.
3. Start searching from the first element.
4. Compare each element with the key.
5. If a match is found, return its index.
6. Otherwise, continue to the next element.
7. Stop when the key is found or the array ends.
8. Return 4 because 50 is found at index 4.

Expected Output

Element found at index: 4

If the key were 60 instead of 50, the last step would be:

Element not found

Time Complexity

- Best Case: $O(1)$: Element is found at the first position.
- Average Case: $O(n)$
- Worst Case: $O(n)$: Element is at the last position or not present.
- Space Complexity: $O(1)$